

Comanda Digital

Sistema de Pedidos para Dark Kitchen

Especificacao de Requisitos de Software (SRS)

Spring Boot

Angular

MySQL

Instituicao: UNASP SP - Centro Universitario Adventista de Sao Paulo

Curso: Analise e Desenvolvimento de Sistemas

Disciplina: Desenvolvimento Full-Stack (G01371.1)

Professor: Thiago Silva

Semestre: 2026/1 - 4 creditos - 80h total

Stack: Java Spring Boot + Angular + MySQL

Execucao: Local (localhost) - deve rodar na maquina do professor

Versao: 2.0 - Fevereiro 2026

Modulo	Descricao	Tipo
Area do Cliente	Cardapio publico, carrinho, cadastro/login, checkout e pedido	Obrigatorio
Gestao de Pedidos	Cozinha recebe pedidos, muda status, acompanha fila	Obrigatorio
Cardapio & Fichas Tecnicas	CRUD de pratos com ingredientes e calculo de custo	Obrigatorio
Fornecedores & Compras	Cadastro, cotacao, pedido de compra e recebimento	Obrigatorio
Controle de Estoque	Entradas, saidas automaticas, alertas de minimo	Obrigatorio
Dashboard	KPIs, top pratos, food cost, alertas	Obrigatorio

O sistema deve rodar 100% local (localhost). O professor vai clonar o repositorio, rodar o projeto e testar. Se nao funcionar, a nota maxima e 4,0 nos sprints.

Sumario

1. O Que Voces Vao Construir

- 1.1 O Cenario: Dark Kitchen
- 1.2 Fluxo Geral do Sistema
- 1.3 Perfis de Usuario

2. Stack e Ambiente

- 2.1 Back-end: Spring Boot
- 2.2 Front-end: Angular
- 2.3 Banco: MySQL
- 2.4 Como Entregar

3. Requisitos Funcionais

- 3.1 Area do Cliente (cardapio, carrinho, pedido)
- 3.2 Cardapio & Fichas Tecnicas (admin)
- 3.3 Pedidos & Cozinha
- 3.4 Fornecedores & Compras
- 3.5 Controle de Estoque
- 3.6 Dashboard
- 3.7 Autenticacao & Usuarios

4. Regras de Negocio

5. Requisitos Nao-Funcionais

6. Banco de Dados (MySQL)

7. Endpoints da API

8. Fluxos do Sistema

9. Criterios de Avaliacao e Sprints

1. O Que Voces Vao Construir

Voces vao construir o **Comanda Digital**, um sistema web completo para uma **Dark Kitchen** - uma cozinha que so vende por delivery, sem salao de restaurante. O sistema tem duas "pontas":

- O **cliente** acessa o cardapio, monta seu pedido, paga e acompanha o status.
- A **cozinha/admin** gerencia pratos, recebe pedidos, controla estoque e fornecedores.

Pensem nisso como um "mini-iFood" do lado do restaurante: o cliente faz o pedido pelo site, a cozinha recebe na tela, prepara e marca como pronto. So que alem disso, o admin tambem controla quanto custa cada prato (ficha tecnica com ingredientes), de onde compra os ingredientes (fornecedores) e quanto tem no estoque.

1.1 O Cenario: Dark Kitchen

Dark Kitchen e uma cozinha industrial que nao tem salao. Ela so existe pra fazer comida e mandar por delivery. Caracteristicas que importam pro sistema:

- **Sem balcao:** O cliente nunca vai na loja. Tudo e pelo site.
- **Margens apertadas:** Se o dono nao souber quanto custa cada prato, quebra. Por isso a ficha tecnica e tao importante.
- **Estoque gira rapido:** Ingredientes estragam. Precisa controlar validade e quantidade.
- **Pedidos em fila:** A cozinha recebe varios pedidos ao mesmo tempo e precisa organizar a producao.

1.2 Fluxo Geral do Sistema

O sistema funciona em dois grandes fluxos que se conectam:

Fluxo do Cliente (quem compra):

- **1:** Cliente acessa o site e ve o cardapio (nao precisa estar logado pra ver)
- **2:** Escolhe os pratos, adiciona ao carrinho com quantidade e observacoes
- **3:** Na hora de finalizar, faz login ou cria uma conta (cadastro simples: nome, email, senha, endereco, telefone)
- **4:** Confirma o pedido (pagamento simulado - nao precisa integrar gateway real)
- **5:** Acompanha o status do pedido: RECEBIDO > CONFIRMADO > EM PREPARO > PRONTO > SAIU PRA ENTREGA
- **6:** Pode ver historico dos seus pedidos anteriores

Fluxo da Cozinha/Admin (quem gerencia):

- **1:** Admin cadastra ingredientes, fornecedores e monta o cardapio com fichas tecnicas
- **2:** Sistema calcula automaticamente o custo de cada prato com base nos ingredientes
- **3:** Quando cliente faz pedido, aparece na tela da cozinha
- **4:** Cozinheiro confirma e muda o status conforme vai preparando
- **5:** Ao confirmar, sistema da baixa automatica no estoque
- **6:** Quando estoque fica baixo, sistema alerta. Admin faz pedido de compra pro fornecedor
- **7:** Ao receber mercadoria, registra entrada no estoque e custo se atualiza

1.3 Perfis de Usuario

Perfil	Quem e	O que faz	Como acessa
CLIENTE	Consumidor final	Ve cardapio, faz pedido, acompanha status, ve historico	Cadastro publico + login
ADMIN	Dono da cozinha	Tudo: cardapio, fichas, fornecedores, estoque, compras, dashboard, usuarios	Login (seed no banco)
GERENTE	Gerente de operacoes	Pedidos, estoque, fornecedores, dashboard	Criado pelo ADMIN
COZINHEIRO	Quem prepara	Ve pedidos pendentes e muda status (CONFIRMADO > EM PREPARO > PRONTO)	Criado pelo ADMIN

Importante: O CLIENTE e um perfil completamente diferente dos outros. Ele tem telas proprias (cardapio publico, carrinho, checkout, meus pedidos). Os outros perfis acessam o painel administrativo. Sao praticamente "dois sistemas" em um.

2. Stack e Ambiente

A stack é **obrigatoria**. Não pode trocar Angular por React, não pode trocar Spring por Node, não pode trocar MySQL por MongoDB. O objetivo é todo mundo aprender a mesma coisa.

2.1 Back-end: Spring Boot

Item	O que usar
Java	17 ou superior
Framework	Spring Boot 3.x (iniciar pelo start.spring.io)
Build	Maven ou Gradle (tanto faz)
Banco	Spring Data JPA + Hibernate (mapeamento automático)
Segurança	Spring Security + JWT
Validação	Bean Validation (@NotBlank, @Email, @Min, etc)
Doc da API	SpringDoc OpenAPI (Swagger) - acessar em <code>/swagger-ui.html</code>
Migrations	Flyway (scripts SQL versionados em <code>db/migration</code>)

Organização de pacotes (obrigatoria):

- controller/ - os endpoints REST (@RestController)
- service/ - lógica de negócio (cálculos, regras, validações)
- repository/ - interfaces JPA (extends JpaRepository)
- model/ ou entity/ - as classes que viram tabelas (@Entity)
- dto/ - objetos de request e response (nunca retornar Entity direto)
- config/ - SecurityConfig, CorsConfig, etc
- exception/ - tratamento de erros (@ControllerAdvice)

2.2 Front-end: Angular

Item	O que usar
Angular	17 ou superior (ng new meu-projeto)
Linguagem	TypeScript (já vem com Angular)
UI	Angular Material, PrimeNG ou Bootstrap - você escolhe
HTTP	HttpClient + interceptor pra colocar o JWT no header
Forms	Reactive Forms (FormGroup, FormArray pra ficha técnica)
Rotas	Angular Router com AuthGuard e RoleGuard
Gráficos	Chart.js (ng2-charts) pro dashboard

2.3 Banco de Dados: MySQL

Item	O que usar
Versao	MySQL 15+ (instalar local ou usar Docker)
Nomes	snake_case pra tabelas e colunas (ex: ficha_tecnica_item)
Migrations	Flyway - criar arquivos V1__create_tables.sql, V2__seed_data.sql, etc
Dados iniciais	Script seed com: 1 admin, categorias, ingredientes e pelo menos 5 pratos de exemplo com ficha tecnica
application.properties	spring.datasource.url=jdbc:mysql://localhost:3306/comanda_digital

2.4 Como Entregar e Como Eu Vou Testar

Eu vou clonar o repositório de vocês, rodar na minha máquina e testar tudo. Se não funcionar, a nota máxima nos sprints é 4,0. Então caprichem no README.

- **Repositorio Git** (GitHub): código, commits frequentes, branches organizados
- **README.md** com instruções CLARAS: pre-requisitos, como criar o banco, como rodar o back, como rodar o front
- **Script de seed** no Flyway com dados de exemplo (admin padrão: admin@email.com / senha123)
- **Collection Postman** exportada com todos os endpoints (opcional mas recomendado)
- **Swagger** funcionando em <http://localhost:8080/swagger-ui.html>
- Tudo rodando em **localhost**: back na 8080, front na 4200, mysql na 3306

3. Requisitos Funcionais

Cada requisito tem um ID (RF-XXX) e prioridade. **Alta = obrigatorio**, Media = importante mas nao reprova, Bonus = pontuacao extra.

3.1 Area do Cliente (Cardapio Publico, Carrinho e Pedido)

Essa e a parte que o **cliente final** usa. Ele acessa o site, ve os pratos disponiveis, monta o carrinho, faz login (ou cria conta) e fecha o pedido. Depois acompanha o status.

ID	Requisito	Descricao	Prior.
RF-001	Cardapio publico	Tela publica (sem login) que lista pratos ativos com foto, nome, descricao, preco e categoria. Filtro por categoria.	Alta
RF-002	Detalhe do prato	Ao clicar no prato, abrir detalhe com descricao completa, tempo de preparo e botao "Adicionar ao Carrinho".	Alta
RF-003	Carrinho de compras	Adicionar pratos com quantidade e observacoes (ex: "sem cebola"). Exibir subtotal por item e total geral. Permitir remover itens e alterar quantidade.	Alta
RF-004	Cadastro de cliente	Tela de cadastro: nome, email, senha, telefone e endereco de entrega. Validar email unico. Senha com BCrypt.	Alta
RF-005	Login do cliente	Login com email + senha. Retornar JWT. Redirecionar pro checkout se veio do carrinho.	Alta
RF-006	Checkout (finalizar pedido)	Exibir resumo do carrinho + endereco de entrega. Botao "Confirmar Pedido". Pagamento simulado (nao precisa gateway real, so marcar como "pago").	Alta
RF-007	Acompanhar pedido	Apos confirmar, exibir status atual do pedido com timeline visual: RECEBIDO > CONFIRMADO > EM_PREPARO > PRONTO > SAIU_ENTREGA.	Alta
RF-008	Meus pedidos	Historico de pedidos do cliente logado. Listar com data, itens, valor total e status. Mais recente primeiro.	Media

3.2 Cardapio & Fichas Tecnicas (Painel Admin)

Aqui o admin monta o cardapio. A parte mais importante e a **ficha tecnica**: cada prato tem uma lista de ingredientes com quantidades. O sistema usa isso pra calcular o custo do prato e dar baixa no estoque.

ID	Requisito	Descricao	Prior.
RF-009	CRUD categorias	Criar, editar, listar e desativar categorias (ex: Lanches, Acai, Bebidas).	Alta
RF-010	CRUD pratos	Nome, descricao, foto (URL), preco de venda, tempo de preparo, categoria, status (ATIVO/INATIVO).	Alta
RF-011	Ficha tecnica	Para cada prato: lista de ingredientes com quantidade, unidade (g, ml, un) e fator de correcao. Informar rendimento (quantas porcoes).	Alta
RF-012	Calculo de custo automatico	$\text{custo} = \text{SUM}(\text{qtd} \times \text{fator_correcao} \times \text{custo_unitario}) / \text{rendimento}$. Calcular no SERVICE do Spring Boot. Exibir na tela em tempo real.	Alta

ID	Requisito	Descricao	Prior.
RF-013	Food cost (%)	food_cost = (custo / preco_venda) x 100. Mostrar com cor: verde (<=30%), amarelo (31-35%), vermelho (>35%).	Alta
RF-014	Modo de preparo	Campo de texto com instrucoes passo a passo.	Media

Exemplo - Ficha Tecnica do Hamburguer Artesanal (preco: R\$ 39,90):

Ingrediente	Qtd	Un	FC	Custo Un.	Custo
Blend bovino	180	g	1.00	R\$ 0,045/g	R\$ 8,10
Pao brioche	1	un	1.00	R\$ 2,80	R\$ 2,80
Queijo cheddar	40	g	1.00	R\$ 0,065/g	R\$ 2,60
Alface	30	g	1.40	R\$ 0,012/g	R\$ 0,50
Tomate	50	g	1.25	R\$ 0,008/g	R\$ 0,50
Molho especial	25	ml	1.00	R\$ 0,032/ml	R\$ 0,80
TOTAL					R\$ 15,30

Rendimento: 1 porcao | Custo: R\$ 15,30 | Food Cost: 38,3% (VERMELHO) | FC = Fator de Correcao (perda no preparo)

3.3 Pedidos & Cozinha

ID	Requisito	Descrição	Prior.
RF-015	Receber pedidos	Quando cliente finaliza pedido, ele aparece na lista de pedidos do painel admin com status RECEBIDO.	Alta
RF-016	Status do pedido	Ciclo: RECEBIDO > CONFIRMADO > EM_PREPARO > PRONTO > SAIU_ENTREGA > FINALIZADO. Também CANCELADO.	Alta
RF-017	Baixa automática de estoque	Ao mudar para CONFIRMADO, dar baixa nos ingredientes automaticamente com base na ficha técnica x quantidade. Tudo no mesmo @Transactional.	Alta
RF-018	Cancelamento com estorno	Cancelar pedido reverte estoque (cria movimentação ESTORNO). Exigir motivo. Após EM_PREPARO, só GERENTE/ADMIN cancela.	Alta
RF-019	Lista de pedidos	Listar com filtros: data, status, canal. Paginação obrigatória (Pageable). Mais recentes primeiro.	Alta
RF-020	Detalhe do pedido	Ver todos os itens, quantidades, observações, dados do cliente, valores e timeline de status.	Media

3.4 Fornecedores & Compras

ID	Requisito	Descrição	Prior.
RF-021	CRUD fornecedores	Razão social, CNPJ (validar), telefone, email, categorias de produtos, status.	Alta
RF-022	Catálogo por fornecedor	Vincular ingredientes a cada fornecedor com preço e unidade de venda. Mesma coisa que dizer "esse fornecedor vende alface a R\$ 0,012/g".	Alta
RF-023	Cotação comparativa	Selecionar ingrediente e ver todos os fornecedores que vendem, com preços lado a lado. Ordenar por preço.	Alta
RF-024	Pedido de compra	Selecionar fornecedor, adicionar itens com qtd. Status: RASCUNHO > ENVIADO > RECEBIDO. Calcular total.	Alta
RF-025	Recebimento	Ao receber mercadoria, registrar. Sistema cria entrada no estoque e atualiza custo unitário do ingrediente.	Alta
RF-026	Histórico de preços	Guardar histórico de preço por ingrediente/fornecedor. Exibir gráfico de evolução (Chart.js).	Media

3.5 Controle de Estoque

ID	Requisito	Descrição	Prior.
RF-027	CRUD ingredientes	Nome, SKU, unidade padrão (g, ml, un, kg, L), estoque mínimo, custo unitário atual, status.	Alta
RF-028	Entrada de estoque	Automática ao receber compra OU manual (ajuste). Registrar quantidade, lote, validade, custo.	Alta

ID	Requisito	Descricao	Prior.
RF-029	Saida automatica	Ao confirmar pedido, baixar ingredientes conforme ficha tecnica. Registrar como movimentacao tipo SAIDA/VENDA.	Alta
RF-030	Saida manual (perdas)	Registrar com motivo obrigatorio: DESPERDICIO, VENCIMENTO, QUEBRA, USO_INTERNO. Enum no back.	Alta
RF-031	Saldo em tempo real	Endpoint que retorna saldo de cada ingrediente (soma entradas - soma saidas). Exibir na tela.	Alta
RF-032	Alertas de minimo	Listar ingredientes abaixo do estoque minimo. Badge vermelho no menu. Card no dashboard.	Alta
RF-033	Historico de movimentacoes	Todas as entradas e saidas com data, tipo, quantidade, motivo, usuario e referencia (pedido ou compra).	Media

3.6 Dashboard

ID	Requisito	Descricao	Prior.
RF-034	KPIs do dia	Cards: faturamento do dia, total de pedidos, ticket medio, food cost medio. Usar queries no repository.	Alta
RF-035	Top 5 pratos	Ranking dos mais vendidos (periodo selecionavel). Grafico de barras com Chart.js.	Alta
RF-036	Alertas de estoque	Lista de ingredientes criticos direto no dashboard.	Alta
RF-037	Vendas por periodo	Grafico de linha com faturamento diario. Tabela resumo.	Media

3.7 Autenticacao & Usuarios

ID	Requisito	Descricao	Prior.
RF-038	Login (todos)	POST /api/auth/login com email+senha, retorna JWT com role no payload. Funciona pra CLIENTE e pra ADMIN/GERENTE/COZINHEIRO.	Alta
RF-039	Cadastro publico (cliente)	POST /api/auth/register - qualquer pessoa cria conta com perfil CLIENTE. Os outros perfis so ADMIN cria.	Alta
RF-040	RBAC (controle por perfil)	4 perfis: ADMIN, GERENTE, COZINHEIRO, CLIENTE. Usar @PreAuthorize no Spring. Guards no Angular.	Alta
RF-041	CRUD usuarios (admin)	Admin cria/edita/desativa usuarios internos (gerente, cozinheiro). Senha com BCrypt.	Alta
RF-042	Interceptor Angular	HttpInterceptor que coloca Authorization: Bearer [token] em toda request. Tratar 401 > redirecionar pra login.	Alta
RF-043	Protecao de rotas Angular	AuthGuard (logado?), RoleGuard (tem permissao?). Cliente nao acessa admin. Cozinheiro nao acessa fornecedores.	Alta

4. Regras de Negocio

Essas regras devem estar no **Service**, nunca no Controller. O Controller so recebe a request, chama o Service e devolve a response.

ID	Regra	Descricao Simples	Onde
RN-01	Prato precisa de ficha	Prato so fica ATIVO se tiver ficha tecnica com pelo menos 1 ingrediente.	PratoService
RN-02	Food cost > 35% = alerta	Se food cost passar de 35%, retornar um aviso no response (nao bloquear, so avisar).	FichaTecnicaService
RN-03	Sem estoque = sem pedido	Se ingrediente de algum prato do pedido tiver estoque insuficiente, retornar 422 dizendo o que falta.	PedidoService
RN-04	Cancelamento tem regra	Antes de EM_PREPARO: qualquer um cancela. Depois: so GERENTE ou ADMIN.	PedidoService
RN-05	Custo atualiza na compra	Quando registra recebimento de compra, custo unitario do ingrediente atualiza pro preco novo.	CompraService
RN-06	Exclusao logica	Nunca deletar registro fisicamente. Usar campo status = INATIVO (soft delete).	Todos
RN-07	CNPJ valido	Validar CNPJ do fornecedor (algoritmo). Pode usar lib ou implementar.	Validator
RN-08	Fator correcao >= 1.0	Na ficha tecnica, fator de correcao nao pode ser menor que 1.0 (validar com @Min).	DTO
RN-09	Cliente so ve pratos ativos	O cardapio publico so exhibe pratos com status ATIVO. Inativos/pausados nao aparecem.	PratoService
RN-10	Email unico	Nao pode ter dois usuarios com mesmo email. Validar no cadastro e retornar 409 se ja existir.	UsuarioService

5. Requisitos Nao-Funcionais (o jeito certo de fazer)

Esses requisitos nao sao "funcionalidades" mas sim **como** o codigo deve ser feito. Se nao seguir, perde ponto mesmo que a tela funcione.

ID	O que	Por que
RNF-01	Senha com BCrypt. NUNCA retornar senha em response.	Seguranca basica. Se eu vir senha no JSON, perde ponto.
RNF-02	JWT com expiracao (8h). Endpoints protegidos com @PreAuthorize.	Aprender controle de acesso real.
RNF-03	CORS configurado pra aceitar localhost:4200.	Sem isso, o Angular nao consegue chamar o Spring.
RNF-04	Controller > Service > Repository. Logica de negocio so no Service.	Separacao de camadas. Fundamento de arquitetura.
RNF-05	Usar DTOs pra request e response. Nunca expor @Entity direto.	Evitar expor dados internos e loops de serializacao.
RNF-06	@ControllerAdvice com tratamento global de erros.	Respostas de erro padronizadas (nao stack trace na cara do cliente).
RNF-07	Flyway pra migrations. Nao usar ddl-auto=create.	Controle de versao do banco. Profissionalismo.
RNF-08	Paginacao (Pageable) em TODAS as listagens.	Nao retornar 10.000 registros de uma vez. Performance basica.
RNF-09	Layout responsivo (desktop + tablet no minimo).	CSS basico. Testar redimensionando o navegador.
RNF-10	Feedback visual: loading, toast de sucesso/erro.	UX minima. O usuario precisa saber o que ta acontecendo.
RNF-11	Swagger funcionando em /swagger-ui.html.	Eu vou usar pra testar a API de voces.
RNF-12	README.md com instrucoes de como rodar.	Se eu nao conseguir rodar, nota maxima 4,0 nos sprints.

6. Banco de Dados (MySQL)

Abaixo estao as tabelas que voces precisam criar via Flyway. Voces devem gerar tambem um **DER visual** (usar dbdiagram.io ou DBeaver) e colocar no repositorio como imagem.

Tabela	Colunas principais
usuario	id BIGINT AUTO_INCREMENT PK, nome, email UNIQUE, senha_hash, perfil (ADMIN/GERENTE/COZINHEIRO/CLIENTE), telefone, endereco, status, created_at, updated_at
categoria	id BIGINT AUTO_INCREMENT PK, nome, descricao, ordem, status, created_at
prato	id BIGINT AUTO_INCREMENT PK, categoria_id FK, nome, descricao, foto_url, preco_venda DECIMAL(10,2), tempo_preparo_min, status (ATIVO/INATIVO/PAUSADO), created_at
ingrediente	id BIGINT AUTO_INCREMENT PK, nome, sku UNIQUE, unidade_padrao (G/ML/UN/KG/L), estoque_minimo DECIMAL, custo_unitario DECIMAL(10,4), status, created_at
ficha_tecnica	id BIGINT AUTO_INCREMENT PK, prato_id FK UNIQUE, rendimento INT, modo_preparo TEXT, created_at
ficha_tecnica_item	id BIGINT AUTO_INCREMENT PK, ficha_tecnica_id FK, ingrediente_id FK, quantidade DECIMAL, unidade, fator_correcao DECIMAL DEFAULT 1.00
fornecedor	id BIGINT AUTO_INCREMENT PK, razao_social, cnpj UNIQUE, telefone, email, status, created_at
fornecedor_produto	id BIGINT AUTO_INCREMENT PK, fornecedor_id FK, ingrediente_id FK, preco DECIMAL(10,4), unidade_venda, UNIQUE(fornecedor_id, ingrediente_id)
pedido_compra	id BIGINT AUTO_INCREMENT PK, fornecedor_id FK, status (RASCUNHO/ENVIADO/RECEBIDO/CANCELADO), valor_total, usuario_id FK, created_at
pedido_compra_item	id BIGINT AUTO_INCREMENT PK, pedido_compra_id FK, ingrediente_id FK, quantidade, preco_unitario, subtotal
pedido	id BIGINT AUTO_INCREMENT PK, cliente_id FK (usuario), status (RECEBIDO/CONFIRMADO/EM_PREPARO/PRONTO/SAIU_ENTREGA/FINALIZADO/CANCELADO), valor_total, endereco_entrega, observacoes, motivo_cancelamento, created_at, updated_at
pedido_item	id BIGINT AUTO_INCREMENT PK, pedido_id FK, prato_id FK, quantidade INT, preco_unitario, observacoes
estoque_movimentacao	id BIGINT AUTO_INCREMENT PK, ingrediente_id FK, tipo (ENTRADA/SAIDA/ESTORNO), quantidade, motivo (COMPRA/VENDA/DESPERDICIO/VENCIMENTO/AJUSTE/ESTORNO), lote, validade DATE, custo_unitario, pedido_compra_id FK NULL, pedido_id FK NULL, usuario_id FK, created_at

Relacionamentos importantes:

- usuario (CLIENTE) 1:N pedido - o cliente faz varios pedidos
- pedido 1:N pedido_item N:1 prato - cada pedido tem varios pratos
- prato 1:1 ficha_tecnica 1:N ficha_tecnica_item N:1 ingrediente
- pedido CONFIRMADO > estoque_movimentacao (baixa automatica via ficha tecnica)
- pedido_compra RECEBIDO > estoque_movimentacao (entrada automatica)
- fornecedor N:N ingrediente (via fornecedor_produto)

7. Endpoints da API REST

Todos retornam JSON. Todos protegidos com JWT exceto os marcados como Publico. O Swagger deve documentar todos automaticamente.

Metodo	Endpoint	O que faz	Quem
POST	/api/auth/login	Login, retorna JWT	Publico
POST	/api/auth/register	Cadastro de cliente	Publico
GET	/api/cardapio	Lista pratos ativos (publico, com filtro por categoria)	Publico
GET	/api/cardapio/{id}	Detalhe do prato (publico)	Publico
POST	/api/pedidos	Criar pedido (carrinho > pedido)	CLIENTE
GET	/api/pedidos/meus	Meus pedidos (historico do cliente logado)	CLIENTE
GET	/api/pedidos/{id}/status	Status atual do pedido	CLIENTE
GET	/api/admin/pedidos	Listar todos os pedidos (paginado, filtros)	ADM/GER/COZ
PATCH	/api/admin/pedidos/{id}/status	Mudar status do pedido	ADM/GER/COZ
PATCH	/api/admin/pedidos/{id}/cancelar	Cancelar pedido com motivo	ADM/GER
CRUD	/api/admin/categorias	GET(lista), POST, PUT/{id}, DELETE/{id}	ADM/GER
CRUD	/api/admin/pratos	GET(paginado), POST, PUT/{id}, DELETE/{id}	ADM/GER
GET	/api/admin/pratos/{id}/custo	Retorna custo calculado + food cost	ADM/GER
GET/POST/PUT	/api/admin/pratos/{id}/ficha	CRUD da ficha tecnica do prato	ADM/GER
CRUD	/api/admin/ingredientes	CRUD de ingredientes	ADM/GER
GET	/api/admin/estoque/saldo	Saldo atual de todos ingredientes	ADM/GER
GET	/api/admin/estoque/alertas	Ingredientes abaixo do minimo	ADM/GER
POST	/api/admin/estoque/movimentacao	Registrar saida manual (perda)	ADM/GER

Metodo	Endpoint	O que faz	Quem
CRUD	/api/admin/fornecedores	CRUD fornecedores + catalogo	ADM/GER
GET	/api/admin/cotacao/{ingredienteId}	Comparativo fornecedores do ingrediente	ADM/GER
CRUD	/api/admin/compras	CRUD pedido de compra	ADM/GER
POST	/api/admin/compras/{id}/receber	Registrar recebimento > entrada estoque	ADM/GER
GET	/api/admin/dashboard/resumo	KPIs: faturamento, pedidos, ticket medio	ADM/GER
GET	/api/admin/dashboard/top-pratos	Top 5 mais vendidos	ADM/GER
CRUD	/api/admin/usuarios	Gerenciar usuarios internos	ADM

8. Fluxos do Sistema

8.1 Fluxo do Cliente (Fazer Pedido)

#	O que o cliente faz	O que o sistema faz
1	Acessa o site	Carrega cardapio publico (GET /api/cardapio). Sem login.
2	Navega por categorias, clica num prato	Exibe detalhe do prato (foto, descricao, preco).
3	Clica "Adicionar ao Carrinho" (qtd + obs)	Salva no carrinho local (Angular - state no service ou localStorage).
4	Clica "Finalizar Pedido"	Se nao logado, redireciona pro login/cadastro. Se logado, vai pro checkout.
5	Faz cadastro (se novo) ou login	POST /api/auth/register ou /login. Retorna JWT. Redireciona pro checkout.
6	Confere resumo e clica "Confirmar"	POST /api/pedidos com itens do carrinho. Back valida estoque. Cria pedido RECEBIDO.
7	Ve tela "Pedido confirmado!"	Exibe status do pedido. Cliente pode acompanhar atualizacoes.

8.2 Fluxo da Cozinha (Processar Pedido)

#	Quem	O que faz	Sistema
1	Gerente	Ve novo pedido na lista	Pedido aparece com status RECEBIDO
2	Gerente	Confirma o pedido	Status > CONFIRMADO. Baixa estoque automatica.
3	Cozinheiro	Inicia preparo	Status > EM_PREPARO
4	Cozinheiro	Termina de fazer	Status > PRONTO
5	Gerente	Entregador levou	Status > SAIU_ENTREGA
6	Sistema	(automatico ou manual)	Status > FINALIZADO

8.3 Fluxo de Compras (Reabastecer Estoque)

#	Acao	Sistema
1	Admin ve alertas de estoque baixo no dashboard	Endpoint /alertas lista ingredientes abaixo do minimo
2	Consulta cotacao do ingrediente	Comparativo de precos entre fornecedores
3	Cria pedido de compra pro fornecedor	Status RASCUNHO > ENVIADO
4	Mercadoria chega, registra recebimento	Status > RECEBIDO. Cria entrada no estoque. Atualiza custo do ingrediente.
5	(automatico)	Food cost de todos os pratos que usam esse ingrediente e recalculado

9. Critérios de Avaliação e Sprints

O trabalho está dividido em 3 Sprints, alinhados com o calendário da disciplina. A composição da nota dos sprints no semestre é:

Avaliação	Data	Peso na Média	O que entregar
Sprint 1	26/03/2026	10%	Interface Angular completa: telas do cliente (cardápio, carrinho, checkout) + telas admin (layout com sidebar, CRUD pratos/categorias). Pode usar dados mockados.
Sprint 2	30/04/2026	20%	API Spring Boot funcionando: auth JWT, CRUD completo de pratos, ingredientes, ficha técnica com cálculo de custo, CRUD fornecedores. Angular consumindo a API (não mais mock).
PVI	28/05/2026	20%	Prova individual (conteúdo da disciplina)
PI	03/06/2026	10%	Prova interdisciplinar
PRJ	17/06/2026	10%	Projeto Integrador (I SIMPEC)
Sprint 3	24/06/2026	30%	Sistema COMPLETO funcionando: área do cliente com pedido, estoque com baixa automática, compras com entrada, dashboard com gráficos. Tudo integrado e rodando.

Média Final = $SP1 \times 0.1 + SP2 \times 0.2 + PVI \times 0.2 + PI \times 0.1 + PRJ \times 0.1 + SP3 \times 0.3$

Os 3 sprints juntos correspondem a **6,0 pontos da média final** ($SP1 = 1,0$ | $SP2 = 2,0$ | $SP3 = 3,0$). Cada sprint é avaliado de 0 a 10, e o peso converte para a nota final. Aprovado com média $\geq 6,0$ e frequência $\geq 75\%$.

9.1 O Que Vale Ponto em Cada Sprint

Cada sprint é avaliado de 0 a 10 conforme os critérios abaixo. Depois o peso transforma na contribuição para a média final.

Sprint 1 - Interface Angular (26/03) - peso 10% = até 1,0 na média	Pontos
Tela de cardápio público com listagem de pratos por categoria	2,0
Tela de carrinho funcional (adicionar, remover, alterar qtd)	2,0
Tela de login e cadastro do cliente	1,5
Layout admin com sidebar e navegação (routing)	1,5
Tela de CRUD de pratos e categorias (formulários com validação)	2,0
Responsivo (desktop + tablet) e feedback visual básico	1,0
TOTAL (nota do sprint, de 0 a 10)	10,0

Sprint 2 - API Spring Boot (30/04) - peso 20% = ate 2,0 na media	Pontos
Spring Security + JWT funcionando (login retorna token, endpoints protegidos)	2,0
CRUD completo pratos + categorias + ingredientes (com DTOs e validacao)	2,0
Ficha tecnica com calculo de custo no Service (formula correta)	2,0
CRUD fornecedores + catalogo de produtos + cotacao comparativa	1,5
Angular consumindo a API real (nao mais mock). Interceptor JWT.	1,5
Swagger funcionando + Flyway com migrations	1,0
TOTAL (nota do sprint, de 0 a 10)	10,0

Sprint 3 - Sistema Completo (24/06) - peso 30% = ate 3,0 na media	Pontos
Fluxo completo do cliente: cardapio > carrinho > cadastro/login > pedido > acompanhar status	2,5
Pedidos com baixa automatica de estoque (via ficha tecnica, @Transactional)	2,0
Estoque: saldo, alertas de minimo, movimentacoes, saida manual	1,0
Compras: pedido de compra > recebimento > entrada no estoque > custo atualiza	1,5
Dashboard com KPIs e graficos (Chart.js)	1,0
Cancelamento com estorno de estoque + regras de permissao	1,0
README funcional + roda na minha maquina sem problemas	1,0
TOTAL (nota do sprint, de 0 a 10)	10,0

Resumo da contribuicao dos sprints na media	Maximo
Sprint 1 (nota x 0.1)	1,0 ponto
Sprint 2 (nota x 0.2)	2,0 pontos
Sprint 3 (nota x 0.3)	3,0 pontos
TOTAL DOS SPRINTS NA MEDIA FINAL	6,0 pontos

9.2 Regras Gerais

- Trabalho **individual ou em dupla** (definir comigo nas primeiras semanas).
- Plagio = zero pra todos os envolvidos, sem recuperacao.
- Se nao rodar na minha maquina, nota maxima e 4,0 nos sprints.** Caprichem no README.
- Seed obrigatorio no Flyway: admin@email.com / senha123 + dados de exemplo.
- Voce deve ser capaz de explicar qualquer trecho do codigo na apresentacao.
- Uso de IA (ChatGPT, Copilot, Claude) e permitido, mas voce precisa entender tudo que esta no projeto.
- Entregas atrasadas: ate 1 semana = -20%, apos 1 semana = nao aceito.

Fim do documento. Comanda Digital - SRS v2.0 - Trabalho Full-Stack - UNASP SP - 1o Sem 2026 - Prof. Thiago Silva